

Employee Lifecycle Attrition Pipeline

Project Status Report

Databricks (Free Edition) · PySpark · Delta Lake · Medallion Architecture · SCD Type 2

Overview

This is an HR analytics data engineering pipeline built on Databricks (Free Edition), PySpark, and Delta Lake. It follows Medallion Architecture (Bronze → Silver → Gold) and uses Slowly Changing Dimension Type 2 (SCD2) to preserve full employee history — promotions, transfers, salary changes, and resignations — rather than overwriting records.

Source data consists of 5 days of daily HR snapshot CSVs (emp_id, name, department, salary, join_date, status). Headcount grows from 200 to 220 across the window, with resignations increasing day over day.

Workspace: Employee_Lifecycle_Attrition_Pipeline, organized into Architecture, Data, Databricks_Notebooks (Data_Ingestion, Bronze_Layer, Silver_Layer, SCD_Type2, Gold_Layer, BI_Data, SQL_Analytics, Dashboard), Output, and a dashboard object.

Pipeline Layers

Layer	Table(s)	Verified State	Purpose
Bronze	employee_data.bronze_employee	1,050 rows	Raw daily CSV snapshots + snapshot_date
Silver	employee_data.silver_employee	594 total / 220 current	SCD Type 2 history of dept, salary, status changes
Gold	gold_avg_tenure, gold_attrition_rate	220 employees	Avg tenure by department, attrition rate
BI	hr_summary	220 employees	Powers the Databricks Dashboard

Bronze Layer

Ingests the five raw daily CSV snapshots as-is and stamps each row with a snapshot_date. Verified at 1,050 rows (5 days × ~210 average headcount) with the correct schema.

Silver Layer — SCD Type 2

For each emp_id, the current day's snapshot is compared against the previous tracked version on department, salary, and status. Unchanged records are left alone; changed records expire the prior row (effective_end_date, is_current = false) and insert a new current row.

- Tracking columns: effective_start_date, effective_end_date, is_current
- Process: rebuild schema → reload Day 1 baseline → diff Bronze dates against processed Silver dates → run the MERGE loop for each remaining day

- Final verified state: 594 total rows, 220 current employees

Gold & BI Layers

Gold produces gold_avg_tenure (average tenure by department) and gold_attrition_rate, plus CSV exports to the Output folder for downstream consumption. BI's hr_summary table powers the Databricks Dashboard.

Data Quality Checks

After any Silver rebuild, Gold and BI must be re-run and reconciled. Both of the following checks return the current employee count and should agree:

- `SELECT SUM(total_employees) FROM employee_data.hr_summary`
- `SELECT SUM(employee_count) FROM employee_data.gold_avg_tenure`

Status: both checks currently return 220 — verified passing.

Known Issues & Fixes

Issue	Root Cause	Fix
Silver row counts wrong after rebuild	Day 1's effective_start_date wasn't stamped correctly, causing it to be reprocessed	Rebuilt schema, reloaded Day 1 baseline with the correct date, then re-ran the merge loop for Days 2–5
NameError: avg_tenure_by_dept not defined	Gold_Layer notebook cells run out of order (write cell executed before the aggregation cell)	Run all Gold_Layer cells top-to-bottom in the same session
Obsolete bronze_day1 table missing snapshot_date	Leftover table from an earlier iteration	Dropped and rebuilt from the corrected ingestion notebook

Current Status

- ✓ Bronze ingestion verified (1,050 rows, correct schema)
- ✓ Silver SCD2 rebuild verified (594 total / 220 current)
- ✓ Gold layer refreshed and re-saved
- ✓ BI hr_summary refreshed and re-saved
- ✓ CSV exports re-run
- ✓ Dashboard refreshed
- ✓ Sanity checks passing (220 / 220)
- Scheduled as an automated Databricks Job
- Automated pre-downstream data-quality checks

Recommended Next Steps

- Schedule as a Job — chain Data_Ingestion (Bronze) → Silver_Layer (SCD2) → Gold_Layer → BI_Data as dependent tasks on the daily snapshot cadence.
- Add automated data-quality checks — a validation task asserting current_employees matches across Gold and BI before the dashboard refreshes, to catch drift automatically rather than manually.
- Guard against out-of-order notebook execution — consider parameterized jobs/tasks in place of manually run notebook cells to avoid NameError-type issues from partial runs.

Tech Stack

- Platform: Databricks (Free Edition, Serverless compute)
- Storage: Delta Lake (Databricks-managed)
- Processing: PySpark
- Pattern: Medallion Architecture + SCD Type 2